StreamCab: Real-Time Taxi Fare Intelligence

Final Project Presentation

Your Name
Group Member 1, Group Member 2, Group Member 3

Course Name / Department

April 17, 2026

Presentation Plan (12 min + 3 min Q&A)

- Overview: problem statement and architecture
- Data: source, collection, schema/metadata
- Process: end-to-end processing inputs and outputs
- Results: dashboard and analytics outcomes
- Key takeaways: what was easy and difficult

1. Overview: Problem Statement

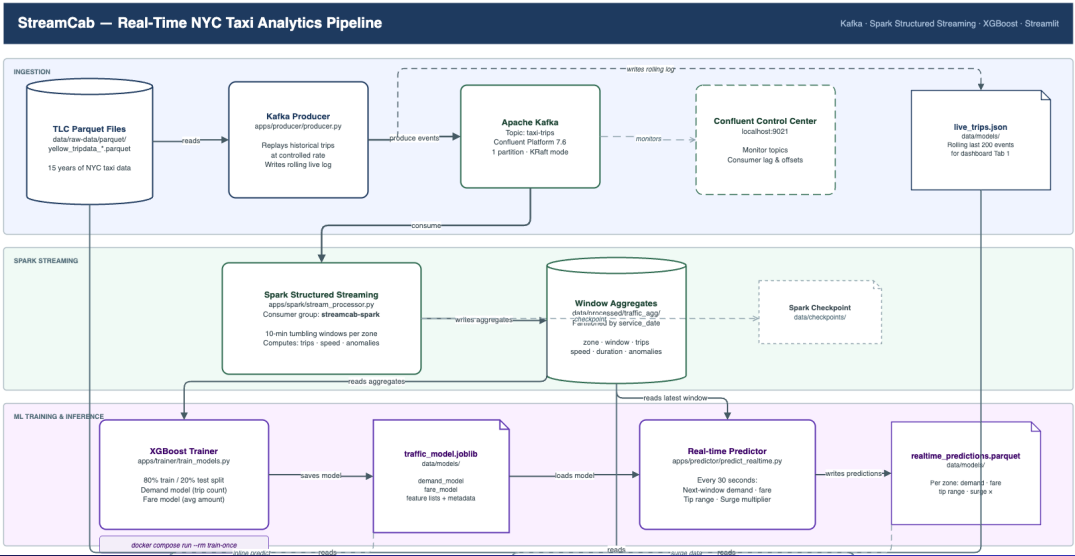
Problem:

- Taxi demand and pricing patterns change quickly by time and location.
- Drivers and operators need near real-time signals for fare and zone activity.

Goal:

- Build an end-to-end streaming analytics pipeline that:
  - ingests NYC taxi trip events,
  - computes rolling traffic aggregates,
  - predicts short-term average fare by pickup zone,
  - serves insights in an interactive dashboard.

1. Overview: System Architecture



Architecture Components

- **Producer + Kafka:** replays parquet trips as event stream.
- **Spark Structured Streaming:** cleans and aggregates by zone/window.
- **Postgres:** stores live trips, aggregate traffic, and predictions.
- **Trainer (XGBoost):** trains fare model from historical data.
- **Predictor:** periodic inference on latest per-zone aggregates.
- **Streamlit Dashboard:** live KPIs, rankings, map, and predictions.

2. Data: Source and Collection

Primary source:

- NYC TLC yellow taxi trip records (public monthly parquet files).

Collection workflow:

- Download selected month range with project script.
- Store raw files under local parquet directory.
- Replay rows as Kafka events for streaming simulation.

Reference data:

- TLC zone centroids file for geospatial zone mapping.

2. Data: Schema and Metadata

Event-level fields (example):

- pickup_datetime, dropoff_datetime
- passenger_count, trip_distance
- pu_location_id, do_location_id
- total_amount, emitted_at

Aggregated output (windowed):

- window_start/window_end, pu_location_id, trip_count
- avg_speed_mph, avg_duration_min, avg_trip_distance
- avg_total_amount, anomaly_count

Prediction output:

- predicted_avg_fare, predicted_tip_low/high, surge_multiplier

3. Process: End-to-End Method

1. Ingest historical parquet data and replay into Kafka.
2. Consume stream and run quality filters (distance, speed, duration bounds).
3. Aggregate by pickup zone and sliding windows in Spark.
4. Upsert aggregates into Postgres for stable serving.
5. Train XGBoost fare model (continued training with batch updates).
6. Score latest per-zone aggregates and write predictions.
7. Render live dashboard with KPIs, rankings, and map views.

3. Inputs and Outputs

Inputs

- Historical parquet trip files
- Zone centroid reference CSV

Intermediate outputs

- Kafka event stream
- Postgres tables: `live_trips`, `traffic_agg`

Final outputs

- Trained model: `traffic_model.joblib`
- Metrics: `metrics.json`
- Dashboard + prediction table: `predictions`

Tools Brief (if not covered in class)

Apache Kafka

- Distributed event streaming platform for decoupled producers/consumers.

Spark Structured Streaming

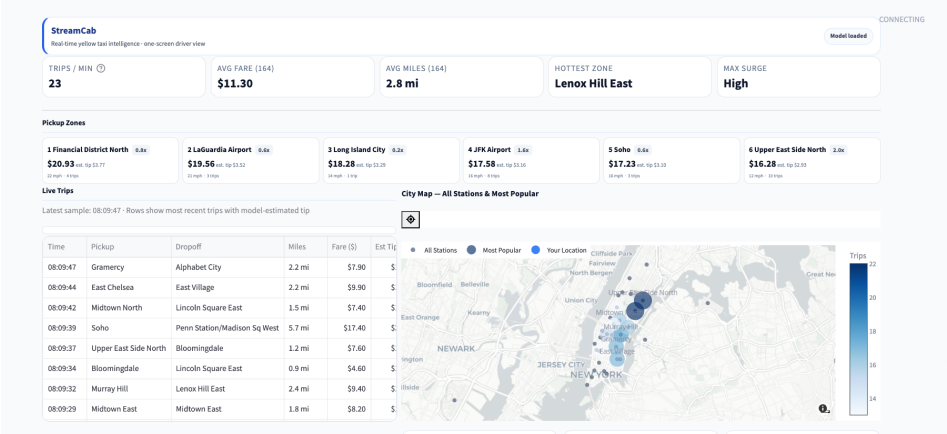
- Micro-batch stream processing using DataFrame API and watermarks/windows.

JDBC / Psycopg

- JDBC: Spark connector for SQL sinks.
- Psycopg: Python Postgres driver for predictor and consumer services.

4. Results: Dashboard and Analytics

- Live trip ingestion and per-minute throughput
- Per-zone fare predictions and surge indicators
- Most active zones and map-based situational awareness



4. Results: Example Metrics Slide

Model	MAE	MAPE (%)
Baseline (zone-hour mean)	1.9891	15.43
XGBoost (test)	0.4851	3.42

Interpretation:

- Significant reduction in absolute and percentage error.
- Streaming aggregates provide robust short-horizon features.

5. Key Takeaways

What was easier than expected

- Containerized local orchestration with Docker Compose.
- Building streaming feature windows with Spark APIs.

What was difficult

- Memory constraints for large-scale training data.
- Exactly-once style behavior across Kafka, Spark, and SQL sink.
- Balancing model quality and runtime latency.

Reproducibility and Submission Checklist

- GitHub repo includes README walkthrough from input to output.
- Public data source link included in README and report.
- Final report includes all required sections and references.
- Group member names and your name included on both deliverables.

Q&A

Thank you.